

Java I/O, File Handling, Streams, Java 8 & Selenium WebDriver

Interview-Oriented Study Notes

Beginner to interview level • Concepts • Examples • Practical scenarios • Frequently asked questions

Study method: Read the concept, run the example, then answer the interview questions without looking at the answers.

Contents

1. File Handling in Java
2. Java I/O
3. Java 8 Features
4. Java Stream API
5. Introduction to Selenium WebDriver
6. Practical Selenium Examples
7. Interview Questions & Answers
8. Coding & Scenario-Based Questions
9. Quick Revision Cheat Sheet

1. File Handling in Java

File handling means creating, reading, writing, updating, and deleting files and directories. Java provides `java.io` and `java.nio.file` for these operations.

1.1 File class

`java.io.File` represents a file or directory path. It can check existence, type, permissions, size, and directory contents. It does not itself read or write file contents.

```
File file = new File("data.txt");
System.out.println(file.exists());
System.out.println(file.getAbsolutePath());
System.out.println(file.length());
```

1.2 Creating, writing and reading a text file

```
File file = new File("data.txt");
try {
    if (file.createNewFile()) {
        System.out.println("File created");
    }
} catch (IOException e) {
    e.printStackTrace();
}

try (FileWriter writer = new FileWriter(file)) {
    writer.write("Hello Java");
} catch (IOException e) {
    e.printStackTrace();
}

try (BufferedReader br = new BufferedReader(new FileReader(file))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

1.3 Important classes

FileReader/FileWriter: character-based text I/O. **FileInputStream/FileOutputStream:** byte-based I/O, useful for binary files. **BufferedReader/BufferedWriter:** buffered access. **Scanner:** convenient simple input.

1.4 Try-with-resources

Try-with-resources automatically closes objects implementing `AutoCloseable`, reducing resource leaks.

```
try (BufferedReader br =
    new BufferedReader(new FileReader("data.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
```

1.5 Modern NIO file API

```
Path path = Paths.get("data.txt");
Files.writeString(path, "Hello NIO");
String text = Files.readString(path);
System.out.println(text);
```

2. Java I/O

Java I/O provides APIs to move data between a program and external resources such as files, memory, console, and streams.

2.1 Byte streams vs Character streams

Type	Base classes	Typical use
Byte streams	InputStream / OutputStream	Images, PDFs, binary data
Character streams	Reader / Writer	Text and character data

2.2 Core hierarchy

```
InputStream -> FileInputStream
OutputStream -> FileOutputStream
Reader -> FileReader
Writer -> FileWriter
```

2.3 Buffered I/O

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("log.txt"))) {
    bw.write("Login successful");
    bw.newLine();
    bw.write("Test completed");
} catch (IOException e) {
    e.printStackTrace();
}
```

2.4 Serialization

Serialization converts an object into a byte stream for storage or transfer. Deserialization reconstructs it. A class normally implements `Serializable`. `transient` excludes a field from default serialization.

```
class Employee implements Serializable {
    private static final long serialVersionUID = 1L;
    String name;
    transient String password;
}
```

2.5 Exceptions

IOException is a checked exception commonly associated with I/O operations. **FileNotFoundException** is a subclass of `IOException`. Use `try-with-resources` to close resources reliably.

3. Java 8 Features

Java 8 added functional programming capabilities and improved collection processing. Interview-relevant features include lambdas, functional interfaces, Stream API, Optional, default/static interface methods, method references, and the Date/Time API.

3.1 Lambda expression

A lambda is a concise implementation of a functional interface.

```
List<String> names = Arrays.asList("Ravi", "Anu", "John");
names.forEach(name -> System.out.println(name));
```

3.2 Functional interface

An interface with exactly one abstract method is a functional interface. Examples include Runnable, Comparator, Predicate, Function, Consumer, and Supplier.

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
Calculator c = (a, b) -> a + b;
System.out.println(c.add(10, 20));
```

3.3 Predicate, Function, Consumer, Supplier

```
Predicate<Integer> even = n -> n % 2 == 0;
Function<String, Integer> length = s -> s.length();
Consumer<String> print = s -> System.out.println(s);
Supplier<Double> random = () -> Math.random();
```

3.4 Method reference

```
names.forEach(System.out::println);
```

3.5 Optional

Optional represents a value that may be absent and encourages explicit handling.

```
Optional<String> name = Optional.ofNullable(getName());
String result = name.orElse("Unknown");
```

3.6 Default and static interface methods

Interfaces can contain default implementations and static utility methods, allowing interfaces to evolve while preserving compatibility.

3.7 Date and Time API

```
LocalDate today = LocalDate.now();
LocalDate nextWeek = today.plusDays(7);
DateTimeFormatter f =
    DateTimeFormatter.ofPattern("dd-MM-yyyy");
System.out.println(today.format(f));
```

4. Java Stream API

A Stream is a sequence of elements supporting declarative processing. A stream does not store data; it processes a source such as a Collection or array.

4.1 Stream pipeline

Typical pipeline: **source** → **intermediate operations** → **terminal operation**. Intermediate operations are generally lazy; a terminal operation triggers processing.

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6);

List<Integer> result = nums.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 10)
    .collect(Collectors.toList());

System.out.println(result); // [20, 40, 60]
```

4.2 Common operations

filter: select • **map**: transform • **flatMap**: flatten • **distinct**: remove duplicates • **sorted**: sort • **limit/skip**: control quantity • **collect**: gather • **reduce**: combine • **count**: count • **findFirst/findAny**: retrieve • **anyMatch/allMatch/noneMatch**: boolean checks.

4.3 map vs flatMap

```
List<String> names = Arrays.asList("Java", "Selenium");
List<Integer> lengths = names.stream()
    .map(String::length)
    .collect(Collectors.toList());

List<List<Integer>> nested =
    Arrays.asList(Arrays.asList(1, 2), Arrays.asList(3, 4));

List<Integer> flat = nested.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());
```

4.4 reduce

```
int sum = nums.stream()
    .reduce(0, Integer::sum);
```

4.5 Grouping

```
Map<Integer, List<String>> byLength = names.stream()
    .collect(Collectors.groupingBy(String::length));
```

4.6 Parallel streams

`parallelStream()` can process elements concurrently, but it is not automatically faster. Be cautious with shared mutable state, ordering requirements, small workloads, and side effects.

5. Introduction to Selenium WebDriver

Selenium is an open-source browser automation ecosystem widely used for web UI testing. WebDriver automates real browsers through browser-specific implementations and the WebDriver standard.

5.1 Typical workflow

Start WebDriver → open URL → locate element → interact → verify → close/quit.

```
WebDriver driver = new ChromeDriver();
driver.get("https://example.com");

WebElement heading = driver.findElement(By.tagName("h1"));
System.out.println(heading.getText());

driver.quit();
```

5.2 Locators

Common locators: id, name, className, tagName, linkText, partialLinkText, cssSelector, xpath. Prefer stable, unique attributes; avoid fragile absolute XPath.

```
driver.findElement(By.id("username"));
driver.findElement(By.name("password"));
driver.findElement(By.cssSelector("#login"));
driver.findElement(By.xpath("//button[@type='submit']"));
```

5.3 findElement vs findElements

findElement() returns the first match and throws NoSuchElementException if none is found. **findElements()** returns a list and returns an empty list when there are no matches.

5.4 WebElement methods

```
element.click();
element.sendKeys("hello");
element.clear();
String text = element.getText();
String value = element.getAttribute("value");
boolean visible = element.isDisplayed();
boolean enabled = element.isEnabled();
boolean selected = element.isSelected();
```

5.5 Waits

Explicit waits are generally preferred for dynamic applications because they wait for a meaningful condition. Implicit waits apply globally to element lookup. Avoid mixing wait strategies carelessly.

```
WebDriverWait wait =
    new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement login = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("login")));
login.click();
```

6. Practical Selenium Examples

6.1 Dropdown

```
WebElement country = driver.findElement(By.id("country"));
Select select = new Select(country);
select.selectByVisibleText("India");
```

6.2 Alert

```
Alert alert = driver.switchTo().alert();
System.out.println(alert.getText());
alert.accept();
```

6.3 Frame

```
driver.switchTo().frame("paymentFrame");
// interact with elements inside the frame
driver.switchTo().defaultContent();
```

6.4 Multiple windows/tabs

```
String parent = driver.getWindowHandle();

for (String handle : driver.getWindowHandles()) {
    if (!handle.equals(parent)) {
        driver.switchTo().window(handle);
        break;
    }
}
```

6.5 Mouse and keyboard Actions

```
Actions actions = new Actions(driver);
actions.moveToElement(menu).click().perform();
actions.keyDown(Keys.CONTROL)
    .sendKeys("a")
    .keyUp(Keys.CONTROL)
    .perform();
```

6.6 JavaScriptExecutor

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].click();", element);
js.executeScript("arguments[0].scrollIntoView(true);", element);
```

6.7 Screenshot

```
File src = ((TakesScreenshot) driver)
    .getScreenshotAs(OutputType.FILE);
Files.copy(src.toPath(), Paths.get("screenshot.png"),
    StandardCopyOption.REPLACE_EXISTING);
```

6.8 Page Object Model

POM separates page locators/actions from test logic. UI changes are then concentrated in page/component classes, improving maintainability.

```
public class LoginPage {
    private WebDriver driver;
    private By username = By.id("username");
    private By password = By.id("password");
    private By login = By.id("login");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void login(String user, String pass) {
        driver.findElement(username).sendKeys(user);
        driver.findElement(password).sendKeys(pass);
        driver.findElement(login).click();
    }
}
```


}

7. Interview Questions & Answers

7.1 File Handling / I/O

Q. What is the difference between FileReader and FileInputStream?

A. FileReader reads character data and is suitable for text. FileInputStream reads raw bytes and is suitable for binary data.

Q. Why use BufferedReader?

A. It buffers character input and provides convenient methods such as readLine(), reducing direct I/O operations.

Q. What is try-with-resources?

A. A construct that automatically closes AutoCloseable resources when the try block completes, including after exceptions.

Q. What is serialization?

A. Converting an object into a byte stream so it can be stored or transferred. Deserialization reverses the process.

Q. What does transient do?

A. A transient field is skipped during default Java serialization.

7.2 Java 8 / Streams

Q. What is a lambda expression?

A. A concise way to provide an implementation of a functional interface.

Q. What is a functional interface?

A. An interface with exactly one abstract method. It can be a target for a lambda expression.

Q. map vs filter?

A. map transforms elements; filter selects elements that satisfy a condition.

Q. What is flatMap?

A. It maps elements to streams and flattens the resulting streams into one stream.

Q. Are streams reusable?

A. No. After a terminal operation, the stream is consumed.

Q. What is lazy evaluation?

A. Intermediate stream operations usually do not execute until a terminal operation is invoked.

Q. When should Optional be used?

A. When absence is meaningful and should be represented explicitly, commonly as an API return type.

Q. What is reduce?

A. A terminal operation that combines elements into a single result, such as a sum or maximum.

7.3 Selenium

Q. What is Selenium WebDriver?

A. A browser automation API commonly used to test web applications.

Q. findElement vs findElements?

A. findElement returns one match or throws NoSuchElementException; findElements returns a list and returns an empty list when there are no matches.

Q. Which locator is best?

A. Prefer stable, unique attributes such as id or dedicated test attributes. CSS/XPath are useful when needed.

Q. Implicit vs explicit wait?

A. Implicit wait applies globally to element lookup. Explicit wait waits for a specified condition at a specific point.

Q. close vs quit?

A. close() closes the current window; quit() ends the WebDriver session and closes its windows.

Q. How do you handle an alert?

A. Use driver.switchTo().alert(), then accept(), dismiss(), getText(), or sendKeys() as appropriate.

Q. How do you handle an iframe?

A. Use switchTo().frame(), interact inside it, then return with defaultContent() or parentFrame().

Q. What is POM?

A. Page Object Model represents pages/components as classes containing locators and actions, keeping tests easier to maintain.

Q. Why are Selenium tests flaky?

A. Timing problems, unstable locators, animations, network delays, stale elements, shared test data, and weak synchronization are common causes.

8. Coding & Scenario-Based Questions

8.1 Find even numbers using streams

```
List<Integer> nums = Arrays.asList(10, 15, 20, 25, 30);
nums.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

8.2 Find duplicate values

```
Set<Integer> seen = new HashSet<>();
Set<Integer> duplicates = nums.stream()
    .filter(n -> !seen.add(n))
    .collect(Collectors.toSet());
```

8.3 Find maximum

```
Optional<Integer> max = nums.stream()
    .max(Integer::compareTo);
```

8.4 Read lines containing a keyword

```
try (BufferedReader br =
    Files.newBufferedReader(Paths.get("app.log"))) {
    br.lines()
        .filter(line -> line.contains("ERROR"))
        .forEach(System.out::println);
} catch (IOException e) {
    e.printStackTrace();
}
```

8.5 Element is sometimes not clickable

Check whether it is present, visible, enabled, covered by another element, inside a frame, or affected by animation. Use a stable locator and an explicit wait such as `elementToBeClickable`. Avoid `Thread.sleep` as the primary synchronization method.

8.6 StaleElementReferenceException

The DOM changed after the `WebElement` reference was obtained. Re-locate the element after the update, wait for the required state, and avoid keeping old references unnecessarily.

8.7 Dynamic XPath

```
By submit = By.xpath(
    "//button[contains(@class, 'submit') and @type='button']");

// Prefer a stable test attribute when available
By submit2 = By.cssSelector("[data-testid='submit']");
```

8.8 Maintainable automation framework

Typical layers: test classes → page/component objects → utilities → test data/configuration → reporting/logging. Add explicit waits, reusable driver management, clean test data, screenshots on failure, and CI execution.

9. Quick Revision Cheat Sheet

File/I/O

File = path/metadata • FileReader/Writer = text • FileInputStream/OutputStream = bytes • BufferedReader = efficient text reading • try-with-resources = automatic close • Path/Files = modern NIO API

Java 8

Lambda = concise behavior • Functional interface = one abstract method • Predicate = boolean test • Function = transform • Consumer = consume • Supplier = provide • Optional = possible absence • Method reference = ::

Streams

filter = select • map = transform • flatMap = flatten • distinct = unique • sorted = order • limit/skip = slice • collect = gather • reduce = combine • count = number • match operations = boolean checks

Selenium

WebDriver = browser automation • By = locator • WebElement = UI element • WebDriverWait = synchronization • Select = HTML select dropdown • switchTo = alert/frame/window • Actions = advanced interactions • JavascriptExecutor = JS execution • POM = maintainability

Final interview tip

For every topic, explain: **what it is** → **why it is used** → **how it works** → **small code example** → **real project scenario** → **common mistake**. Practice writing stream pipelines, file-reading code, XPath/CSS locators, waits, frames, windows, alerts, and POM classes from memory.